

# Dokumentacja Techniczna Interfejsu Użytkownika TipJar+

## 1.0 Wprowadzenie i Filozofia Projektu

Niniejszy dokument stanowi fundamentalny przewodnik techniczny dla deweloperów front-end pracujących nad platformą TipJar+. Jego celem jest zdefiniowanie i ujednolicenie standardów implementacyjnych, architektonicznych i wizualnych dla wszystkich komponentów interfejsu użytkownika. Dokumentacja ta ma zapewnić spójność kodu, najwyższą jakość wykonania oraz pełną zgodność z założeniami projektu. Strategiczne znaczenie ma dla nas połączenie intuicyjnej prostoty, znanej z aplikacji Web2, z innowacyjnością i wydajnością technologii Web3, co pozwoli obniżyć barierę wejścia dla szerokiego grona użytkowników i zbudować zaufanie do platformy od pierwszego kontaktu.

Kluczowe cele tej dokumentacji obejmują:

- Ujednolicenie Standardów Kodu:** Dokument ten standaryzuje implementację komponentów, definiując stos technologiczny, strukturę kodu i najlepsze praktyki. Dzięki temu kod staje się bardziej przewidywalny, łatwiejszy w utrzymaniu i skalowalny, co jest kluczowe dla długoterminowego rozwoju platformy.
- Zapewnienie Dostępności (WCAG):** Dostępność cyfrowa jest dla nas nienegocjowalnym filarem projektu. Wszystkie komponenty i widoki muszą być projektowane i implementowane zgodnie z wytycznymi WCAG 2.1 na poziomie AA, aby zapewnić równy dostęp dla wszystkich użytkowników, w tym osób z niepełnosprawnościami.
- Spójność Doświadczenia Użytkownika (UX):** Ścisłe przestrzeganie wytycznych dotyczących systemu designu, interakcji i layoutu gwarantuje, że cała platforma będzie postrzegana jako spójna, profesjonalna i intuicyjna. Konsekwencja w UX buduje zaufanie i skraca krzywą uczenia się aplikacji.
- Skalowalność i Wydajność:** Przyjęte rozwiązania architektoniczne i technologiczne zostały wybrane z myślą o obsłudze rosnącej bazy użytkowników, docelowo sięgającej 100 milionów. Optymalizacja wydajności jest priorytetem na każdym etapie implementacji, od renderowania komponentów po animacje.

Poniższe sekcje szczegółowo omawiają fundamenty technologiczne oraz specyfikacje poszczególnych komponentów, które razem tworzą solidną podstawę do realizacji tych ambitnych celów.

## 2.0 Podstawowe Zasady i Stos Technologiczny

Strategiczny wybór spójnego stosu technologicznego oraz konsekwentne przestrzeganie kluczowych zasad architektonicznych to fundamenty, na których opiera się sukces projektu TipJar+. Poniższe technologie i dyrektywy zostały starannie dobrane, aby zapewnić budowę wydajnej, skalowalnej i łatwej w utrzymaniu aplikacji, która sprosta oczekiwaniom zarówno użytkowników, jak i deweloperów.

| Technologia  | Rola w Projekcie i Uzasadnienie                                                                                                                                                                                                                                                                                                                    |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| React        | Główna biblioteka do budowy interfejsu użytkownika. Jej komponentowa architektura doskonale wspiera tworzenie reużywalnych i zarządzalnych elementów UI, co jest kluczowe dla utrzymania spójności i przyspieszenia procesu deweloperskiego w projekcie skalującym się do 100 milionów użytkowników.                                               |
| TypeScript   | Zapewnia statyczne typowanie, co znacząco podnosi bezpieczeństwo i jakość kodu. Umożliwia wczesne wykrywanie błędów, poprawia czytelność oraz ułatwia refaktoryzację i pracę zespołową nad złożoną bazą kodu.                                                                                                                                      |
| Tailwind CSS | Framework CSS oparty na podejściu "utility-first". Umożliwia szybkie prototypowanie i budowanie w pełni niestandardowych interfejsów bezpośrednio w kodzie JSX, co eliminuje konieczność przełączania kontekstu między plikami CSS a JSX i przyspiesza development. Jego filozofia mobile-first idealnie wpisuje się w nasze założenia projektowe. |

| Technologia   | Rola w Projekcie i Uzasadnienie                                                                                                                                                                                                                                                               |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Framer Motion | Biblioteka do implementacji zaawansowanych animacji i mikrointerakcji. Została wybrana ze względu na swoją wydajność, deklaratywne API oraz doskonałą integrację z ekosystemem React. Pozwala na tworzenie płynnych i angażujących doświadczeń bez negatywnego wpływu na wydajność aplikacji. |

## Kluczowe Zasady Architektoniczne

- Podejście Mobile-First:** Każdy komponent i widok musi być projektowany i implementowany w pierwszej kolejności z myślą o urządzeniach mobilnych. W praktyce oznacza to, że domyślne klasy w Tailwind CSS (np. `flex-col`, `w-full`) definiują wygląd na najmniejszych ekranach. Style dla większych urządzeń (tabletów, desktopów) są dodawane za pomocą responsywnych wariantów, takich jak `md:flex-row` czy `lg:w-1/2`. Zasada ta jest nienegocjowalna, a jej prawidłowa aplikacja, demonstrowana w layoutcie `CreatorDashboard.tsx` (Sekcja 4.4), będzie głównym punktem oceny podczas code review.
- Dark Mode by Default:** Ciemny motyw jest domyślnym i jedynym trybem wizualnym dla MVP. Decyzja ta podyktowana jest zarówno estetyką marki, która opiera się na głębokich odcieniach turkus i fioletu, jak i komfortem użytkownika. Ciemne tło minimalizuje zmęczenie oczu, szczególnie podczas dłuższego korzystania z aplikacji, i pozwala na lepsze odwzorowanie cieni i gradientów, co jest kluczowe dla naszej tożsamości wizualnej.
- Dostępność jako Priorytet (WCAG 2.1 AA):** Każdy implementowany komponent musi bezwzględnie spełniać kryteria dostępności określone w standardzie WCAG 2.1 na poziomie AA. Obejmuje to m.in. odpowiedni kontrast kolorów, semantyczną strukturę HTML, pełną obsługę z klawiatury, widoczne stany focus oraz dostarczanie alternatyw tekstowych dla treści nietekstowych. Szczegółowe wytyczne dla poszczególnych elementów znajdują się w ich opisach w sekcji 4.0.

Po zdefiniowaniu fundamentów technologicznych i architektonicznych, przechodzimy do omówienia konkretnych elementów wizualnych, które tworzą unikalną tożsamość wizualną marki TipJar+.

## 3.0 System Design: Typografia, Kolory i Ikony

Spójny i przemyślany system wizualny jest niezbędny do budowania zaufania, profesjonalnego wizerunku i rozpoznawalności marki TipJar+. Poniższe wytyczne dotyczące typografii, palety kolorów i ikon są obligatoryjne dla wszystkich tworzonych komponentów i widoków, zapewniając jednolitość doświadczenia użytkownika na całej platformie.

### 3.1 Typografia

W celu zapewnienia czytelności i unikalnego charakteru wizualnego, platforma opiera się na dwóch starannie dobranych krojach pisma.

| Element          | Krój Pisma    | Waga (Weight)  | Zastosowanie                                                                                                      |
|------------------|---------------|----------------|-------------------------------------------------------------------------------------------------------------------|
| Nagłówki (H1-H3) | Mukta Malar   | Bold/SemiBold  | Główne tytuły sekcji, slogany (np. na Landing Page), kluczowe komunikaty i wyróżnione elementy tekstowe.          |
| Tekst Podstawowy | IBM Plex Sans | Regular/Medium | Paragrafy, opisy, etykiety formularzy, tekst w przyciskach oraz wszystkie pozostałe elementy tekstowe interfejsu. |

### 3.2 Paleta Kolorów

Paleta kolorów została zaprojektowana tak, aby budować nowoczesny, profesjonalny i godny zaufania wizerunek, jednocześnie przyciągając uwagę do kluczowych akcji.

- Tło Główne (Ciemny Turkus):** Kody takie jak #003737 i #002B2B stanowią bazę wizualną aplikacji. Ten głęboki, stonowany kolor buduje zaufanie i profesjonalizm, zmniejsza zmęczenie oczu i stanowi doskonałe tło dla jaśniejszych elementów.

- **Kolor Akcentowy (Złoty):** Odcień zbliżony do #FFD700 jest zarezerwowany dla najważniejszych przycisków Call-To-Action (np. “Wesprzyj”) oraz ikon. Jego symbolika jest kluczowa – złoty kolor symbolizuje wartość i nagrodę, bezpośrednio nawiązując do idei wsparcia finansowego.
- **Tekst (Biały / Jasnoszary):** Czysta biel (#FFFFFF) jest używana dla nagłówków, natomiast lekko złamana biel (#F0F0F0) dla dłuższych bloków tekstu, aby zapewnić wysoki kontrast i komfort czytania na ciemnym tle.
- **Akcent Dodatkowy (Fiolet):** Odcienie takie jak #1a001f (w gradientach) czy #6B46C1 są używane oszczędnie do podkreślenia funkcji związanych ze światem Web3 (odznaki NFT, elementy DAO). Fiolet dodaje paletę powiewu nowości i głębi, kojarząc się z innowacyjnością branży blockchain.
- **Kolory Statusów:** Standardowe kolory są używane do komunikowania stanu systemu: zielony dla operacji zakończonych sukcesem, żółty dla ostrzeżeń i informacji wymagających uwagi, a czerwony dla błędów i akcji krytycznych.

### 3.3 Ikony

W interfejsie stosowany jest jednolity, minimalistyczny styl ikon liniowych (outline). Stosowanie ikon z jednego, zatwierdzonego zestawu (np. Heroicons) jest obligatoryjne, aby uniknąć wizualnego chaosu wynikającego z mieszania różnych stylów (np. grubość kreski, zaokrąglenia).

- **Styl:** Wszystkie ikony muszą mieć styl “outline” (konturowy).
- **Kolorystyka:** Ikony są zazwyczaj białe lub jasnoszare. Ikony akcentowe, związane z kluczowymi akcjami (np. napiwek) lub funkcjami Web3, mogą przyjmować odpowiednio kolor złoty lub fioletowy.
- **Dostępność:** Jest bezwzględnym wymogiem, aby każda ikona interaktywna (np. przycisk-ikona) posiadała tekstową etykietę (`aria-label`) opisującą jej funkcję dla użytkowników technologii asystujących. Ikony czysto dekoracyjne muszą być ukryte przed czytnikami ekranu (`aria-hidden="true"`).

Po omówieniu fundamentalnych zasad wizualnych, dokument przechodzi do szczegółowej analizy architektury i implementacji kluczowych komponentów, które stanowią rdzeń aplikacji TipJar+.

## 4.0 Architektura i Implementacja Kluczowych Komponentów

Ta sekcja stanowi rdzeń niniejszej specyfikacji technicznej. Znajdują się w niej szczegółowe opisy budowy, logiki oraz zasad implementacji najważniejszych widoków i komponentów interaktywnych w TipJar+. Każdy podrozdział zawiera analizę celów UX, proponowany layout, fragment kodu implementacyjnego wraz z objaśnieniem oraz krytyczne wytyczne dotyczące dostępności.

### 4.1 Strona Główna (Landing Page)

Strona główna (Landing Page) pełni strategiczną rolę wizytówki platformy. Jej nadrzędnym celem jest skuteczna konwersja dwóch kluczowych grup docelowych: twórców i fanów. Musi ona w klarowny i przekonujący sposób komunikować unikalną propozycję wartości TipJar+, budując od pierwszych sekund wrażenie profesjonalizmu, zaufania i innowacyjności.

Struktura layoutu strony głównej jest podzielona na logiczne sekcje, prowadzące użytkownika przez narrację o platformie:

1. **Nagłówek (Hero):** Pierwsza, widoczna bez przewijania sekcja. Zawiera chwytliwy slogan, krótkie wyjaśnienie idei platformy oraz dwa kluczowe przyciski Call-To-Action (CTA): “Założ profil twórcy” oraz “Znajdź twórcę”.
2. **“Jak to działa”:** Zwięzła sekcja przedstawiająca kluczowe korzyści platformy (niskie prowizje, globalny zasięg, szybkość) w formie ikon z krótkimi opisami. Ma na celu edukację użytkownika w przystępny sposób.
3. **Sekcja dla Twórców:** Dedykowany segment komunikujący korzyści płynące z dołączenia do TipJar+ z perspektywy twórcy. Musi zawierać dodatkowy przycisk CTA prowadzący do rejestracji.
4. **Sekcja dla Fanów:** Część skierowana do osób wspierających, podkreślająca prostotę i szybkość udzielania napiwków, w tym możliwość wsparcia bez konieczności zakładania konta.
5. **Stopka (Footer):** Zawiera standardowe linki (Regulamin, Polityka Prywatności, Kontakt) oraz elementy budujące wiarygodność, takie jak informacje o partnerach i prawa autorskie.

### Implementacja Techniczna (LandingPage.tsx)

```

import React from 'react';
import { motion } from 'framer-motion';

export function LandingPage() {
  return (
    <div className="min-h-screen bg-gradient-to-br from-[#003737] to-[#1a001f] text-white">
      {/* Górna nawigacja */}
      <header className="flex items-center justify-between p-4">
        <h1 className="text-2xl font-bold font-display">TipJar+</h1>
        <nav>
          <a href="/login" className="mr-4 hover:underline">Zaloguj</a>
          <a href="/signup" className="px-4 py-2 bg-yellow-500 text-gray-900 font-semibold rounded focus:outline-none focus:ring-2 focus:ring-offset-2 focus:ring-yellow-400">
            Zarejestruj się
          </a>
        </nav>
      </header>

      {/* Sekcja Hero */}
      <main>
        <section className="text-center py-16 px-4">
          <h2 className="text-4xl font-display font-bold mb-4">
            Wspieraj ulubionych twórców bez granic!
          </h2>
          <p className="text-lg mb-8 font-sans">
            Globalna platforma napiwków Web3 łatwa w użyciu jak Web2.
          </p>
          <div className="space-x-4">
            <motion.a
              whileHover={{ scale: 1.05 }}
              href="/signup?role=creator"
              className="bg-teal-500 hover:bg-teal-600 text-white font-semibold py-3 px-6 rounded">
                Załóż profil twórcy
              </motion.a>
            <motion.a
              whileHover={{ scale: 1.05 }}
              href="/creators"
              className="bg-purple-600 hover:bg-purple-700 text-white font-semibold py-3 px-6 rounded">
                Znajdź twórcę
              </motion.a>
          </div>
        </section>
        {/* Sekcja korzyści, sekcje Twórcy/Fani - skrócone dla czytelności */}
      </main>
    </div>
  );
}

```

```

<section className="px-4 py-8">
  {/* ...ikony i opisy korzyści... */}
</section>
<section className="px-4 py-8 bg-[#002B2B]">
  {/* ...sekcja dla Twórców... */}
</section>
<section className="px-4 py-8">
  {/* ...sekcja dla Fanów... */}
</section>
</main>

{/* Stopka */}
<footer className="bg-[#002B2B] text-sm text-gray-300 p-4 text-center">
  © 2025 TipJar+ - <a href="/privacy" className="underline hover:text-gray-100">Polityka prywatności</a>
</footer>
</div>
);
}

```

- **Struktura Komponentu:** Komponent jest logicznie podzielony na semantyczne sekcje: `<header>`, `<main>` i `<footer>`, co poprawia czytelność kodu i dostępność. Main zawiera poszczególne sekcje treści.
- **Stylizacja (Tailwind CSS):** Wykorzystano gradient tła (`bg-gradient-to-br from-[#003737] to-[#1a001f]`) dla nadania głębi i nowoczesnego wyglądu. Klasy są zgodne z podejściem `mobile-first` – domyślnie elementy układają się w jednej kolumnie. Przyciski CTA mają wyróżniające kolory z palety marki, aby przyciągać uwagę.
- **Interaktywność (Framer Motion):** Zastosowanie komponentu `motion.a` z właściwością `whileHover={{ scale: 1.05 }}` wprowadza subtelny mikrointerakcję, która wizualnie potwierdza interakcję użytkownika z przyciskiem, zachęcając do kliknięcia.

## Wytyczne Dostępności (WCAG)

- **Kontrast:** Wszystkie teksty (np. biały na tle gradientu) muszą spełniać minimalny współczynnik kontrastu 4.5:1 (WCAG AA). Jest to kluczowe dla użytkowników z wadami wzroku.
- **Teksty Alternatywne:** Wszelkie obrazy i ilustracje o znaczeniu informacyjnym muszą posiadać opisowy atrybut `alt`. Grafiki czysto dekoracyjne powinny mieć pusty atrybut `alt=""`, aby były ignorowane przez czytniki ekranu.
- **Semantyczna Struktura:** Wymagane jest zachowanie logicznej hierarchii nagłówków. Dopuszczalne są dwie struktury: główny slogan jako `<h1>` lub logo w nagłówku jako `<h1>`, a slogan jako `<h2>`. Wybór zależy od kontekstu implementacji, ale hierarchia musi być zachowana.
- **Widoczny Fokus Klawiatury:** Wszystkie elementy interaktywne (linki, przyciski) muszą mieć wyraźnie widoczny styl po uzyskaniu fokusu za pomocą klawiatury (np. `focus:ring`). Zapewnia to, że użytkownicy nawigujący bez myszy zawsze wiedzą, gdzie się znajdują.
- **Link "Przejdź do treści":** W celu ułatwienia nawigacji klawiaturą, obligatoryjne jest zaimplementowanie ukrytego linku "Przejdź do treści" (skip to content) jako pierwszego elementu w nawigacji. Umożliwia on użytkownikom technologii asystujących szybkie ominięcie bloku menu i przejście bezpośrednio do głównej zawartości strony (`<main>`).
- **Responsywność:** Layout musi poprawnie adaptować się do różnych rozmiarów ekranu, zapewniając czytelność i funkcjonalność bez konieczności przewijania w poziomie, nawet przy znacznym powiększeniu tekstu.

## 4.2 Onboarding Twórcy (Creator Onboarding)

Proces onboardingu jest kluczowym momentem w cyklu życia twórcy. Jego celem jest maksymalne uproszczenie startu i zminimalizowanie ryzyka porzucenia procesu. Interfejs musi być przyjazny i prowadzić użytkownika za rękę. Fundamentalną decyzją strategiczną jest **nie wymaganie weryfikacji KYC na starcie**. Pozwalamy twórcom najpierw zobaczyć korzyści (np. otrzymać pierwsze napiwki), a formalności dopełnić dopiero przy próbie wypłaty środków. Takie podejście drastycznie obniża tarcie i zapobiega rezygnacji na wczesnym etapie.

Proces został podzielony na 4 intuicyjne kroki, prezentowane w formie kreatora (wizarda):

1. **Uzupełnienie profilu:** Użytkownik podaje podstawowe dane publiczne (nazwa, bio, kategoria, awatar). Celem jest szybkie stworzenie atrakcyjnej wizytówki.
2. **Konfiguracja płatności:** Krok czysto informacyjny, w którym komunikujemy, że portfel USDC został automatycznie utworzony i jest gotowy do przyjmowania napiwków. Eliminuje to barierę techniczną.
3. **Porada integracyjna:** Dostarczamy twórcom gotowy do skopiowania link do jego publicznego profilu, zachęcając do natychmiastowego udostępnienia go fanom.
4. **Powitanie:** Ekran potwierdzający pomyślne zakończenie procesu, z wyraźnym przyciskiem przekierowującym do Panelu Twórcy.

## Implementacja Techniczna (CreatorOnboarding.tsx)

```
import React, { useState } from 'react';

export function CreatorOnboarding() {
  const [step, setStep] = useState(1);

  const nextStep = () => setStep(prev => Math.min(prev + 1, 4));
  const prevStep = () => setStep(prev => Math.max(prev - 1, 1));

  return (
    <div className="max-w-xl mx-auto p-4 text-white">
      {/* Pasek postępu */}
      <div className="mb-6">
        <div className="bg-gray-700 rounded-full h-2">
          <div
            style={{ width: `${(step / 4) * 100}%` }}
            className="bg-teal-500 h-2 rounded-full transition-width
duration-300">
          </div>
        </div>
        <p className="text-center mt-2">Krok {step} z 4</p>
      </div>

      {/* Krok 1: Uzupełnienie profilu */}
      {step === 1 && (
        <div>
          <h2 className="text-xl font-bold mb-4">Uzupełnij profil
Twórcy</h2>
          <label className="block mb-2">Nazwa Twórcy:
            <input
              type="text"
              className="w-full p-2 rounded bg-gray-800 border border-gray-
600"
```

```

        placeholder="Twoja nazwa lub pseudonim" />
    </label>
    <label className="block mb-2">Opis:
        <textarea
            className="w-full p-2 rounded bg-gray-800 border border-gray-
600"
            placeholder="Krótki opis Ciebie i Twojej twórczości..."
            rows={3}>
        </textarea>
    </label>
</div>
)}}

{/* Krok 2: Konfiguracja płatności */}
{step === 2 && (
    <div>
        <h2 className="text-xl font-bold mb-4">Konfiguracja płatności</h2>
        <p className="mb-4">Twój portfel USDC został <strong>automatycznie
utworzony</strong>.</p>
        <p className="mb-4 text-sm text-gray-300">
            Możesz od razu przyjmować napiwki!
        </p>
    </div>
)}}

{/* Krok 3: Udostępnienie profilu */}
{step === 3 && (
    <div>
        <h2 className="text-xl font-bold mb-4">Udostępnij swój profil</h2>
        <p className="mb-4">Udostępnij fanom ten link do wsparcia:</p>
        <div className="bg-gray-800 p-3 rounded mb-4">
            <code className="break-
all">https://tipjar.plus/twoja_nazwa</code>
        </div>
        <button className="px-3 py-2 bg-gray-700 rounded text-sm mb-4">
            📋 Kopiuj link
        </button>
    </div>
)}}

{/* Krok 4: Powitanie */}
{step === 4 && (
    <div>
        <h2 className="text-xl font-bold mb-4">Twój profil jest gotowy!
</h2>
        <p className="text-sm text-gray-400">Możesz teraz przejść do
swojego panelu i zobaczyć statystyki.</p>
    </div>
)}}

```

```

    { /* Nawigacja Wstecz/Dalej */}
    <div className="mt-6 flex justify-between">
      <button
        onClick={prevStep}
        disabled={step === 1}
        className="px-4 py-2 bg-gray-600 rounded disabled:opacity-50">
        Wstecz
      </button>
      {step < 4 ? (
        <button
          onClick={nextStep}
          className="px-4 py-2 bg-teal-600 hover:bg-teal-700 rounded">
          Dalej
        </button>
      ) : (
        <button
          onClick={() => { /* zakończ onboarding */}}
          className="px-4 py-2 bg-yellow-500 hover:bg-yellow-600 text-
gray-900 rounded">
          Przejdź do Panelu Twórcy
        </button>
      )}
    </div>
  </div>
);
}

```

- **Zarządzanie Stanem:** Do zarządzania aktualnym krokiem kreatora wykorzystano prosty hook `useState`. Zmienna `step` przechowuje numer bieżącego kroku (1-4), a jej zmiana wywołuje rerender komponentu i wyświetlenie odpowiedniej treści.
- **Logika Nawigacji:** Funkcje `nextStep` i `prevStep` w sposób kontrolowany modyfikują stan `step`. Renderowanie przycisku finalizującego ("Przejdź do Panelu Twórcy") jest warunkowe i zależy od tego, czy użytkownik znajduje się na ostatnim, czwartym kroku procesu.
- **Komponenty UI:** Kluczowe elementy interfejsu to wizualny pasek postępu, którego szerokość jest dynamicznie obliczana na podstawie aktualnego kroku (`step / 4`), oraz warunkowe renderowanie sekcji dla każdego z czterech kroków. Pola formularza są jednolicie ostylowane, aby zachować spójność.

## Mikrointerakcje i Animacje

- **Animowany Pasek Postępu:** Zmiana szerokości paska postępu musi być animowana (np. za pomocą CSS `transition`), aby płynnie sygnalizować postęp i motywować użytkownika.
- **Przejścia Między Krokami:** Zaleca się użycie `AnimatePresence` z `Framer Motion`, aby stworzyć płynne przejścia (np. "slide transition") między kolejnymi krokami. Taka animacja poprawia orientację użytkownika w procesie i sprawia, że interfejs jest bardziej dynamiczny.

## Wytyczne Dostępności (WCAG)

- **Etykiety Formularzy:** Każde pole `input` i `textarea` musi być jednoznacznie powiązane z widoczną etykietą `<label>`. Zapewnia to, że użytkownicy czytników ekranu wiedzą, jakie dane należy wprowadzić w danym polu.



- **Komunikaty o Błędach:** Wszelkie błędy walidacji (np. przekroczenie limitu znaków) muszą być komunikowane w formie tekstowej, a nie tylko za pomocą koloru. Komunikaty te powinny być powiązane z odpowiednim polem.
- **Zarządzanie Fokusem:** Po przejściu do nowego kroku, fokus klawiatury musi być automatycznie przeniesiony na pierwszy interaktywny element w nowym widoku. Zapobiega to "gubieniu się" fokusu i usprawnia nawigację.
- **Obsługa Klawiatury:** Cały proces onboardingu musi być w pełni możliwy do ukończenia wyłącznie za pomocą klawiatury, włączając w to nawigację między krokami i interakcję z polami formularza.

### 4.3 Publiczny Profil Twórcy

Publiczny Profil Twórcy jest centralnym punktem interakcji między fanem a twórcą. Pełni on podwójną rolę: jest stroną wizerunkową, która buduje markę osobistą twórcy, oraz stroną transakcyjną, której nadrzędnym celem jest maksymalizacja konwersji, czyli finansowego wsparcia. Jego projekt musi być przejrzysty, wiarygodny i bezpośrednio prowadzić do akcji.

Kluczowe elementy UI profilu:

- **Nagłówek profilu:** Zawiera esencjonalne informacje: awatar, nazwę twórcy i kategorię jego działalności. Tworzy pierwsze wrażenie i pozwala na szybką identyfikację.
- **Opis (bio):** Krótka sekcja tekstowa, w której twórca może się przedstawić. To miejsce na budowanie więzi z odbiorcami.
- **Przycisk "Wesprzyj":** Najważniejszy element interaktywny na stronie. Musi być duży, wyraźnie widoczny i umieszczony w strategicznym miejscu, aby był zawsze łatwo dostępny.
- **Sekcja "Fan Wall":** Wyświetla listę najbardziej zaangażowanych fanów (np. posiadaczy odznak NFT). Jest to potężny element społecznościowy (social proof), który buduje zaufanie i zachęca innych do wsparcia.
- **Lista ostatnich napiwków:** Dynamicznie aktualizowana lista ostatnich wpłat, która pokazuje, że profil jest aktywny i inni użytkownicy wspierają twórcę, co dodatkowo motywuje do działania.

### Implementacja Techniczna (CreatorProfile.tsx)

```
import React from 'react';
import { motion } from 'framer-motion';

export function CreatorProfile() {
  const topFans = ["AnnaK", "Jan99", "CryptoFan"]; // przykładowe dane top fanów

  return (
    <div className="max-w-2xl mx-auto p-4 text-white">
      {/* Nagłówek profilu */}
      <header className="flex items-center mb-4">
        
        <div>
          <h1 className="text-2xl font-display font-bold">Nazwa Twórcy</h1>
          <p className="text-gray-300">Kategoria / Motto twórcy</p>
        </div>
      </header>

      {/* Opis bio */}
      <p className="mb-6">
```

```

    Krótki opis twórcy... (bio maks kilka zdań, linki social media itp.)
  </p>

  {/* Przycisk wsparcia - może być 'sticky' na mobile */}
  <motion.button
    whileHover={{ scale: 1.05 }}
    className="w-full bg-yellow-500 text-gray-900 font-semibold py-3
rounded mb-6 sticky bottom-4 md:static">
    ❤️ Wesprzyj Twórcę
  </motion.button>
  {/* Po kliknięciu otworzy się TipModal */}

  {/* Top Fani sekcja */}
  <section className="mb-6">
    <h2 className="font-semibold mb-2">Top Fani:</h2>
    <ul className="bg-gray-800 p-4 rounded">
      {topFans.map(fan => (
        <li
          key={fan}
          className="flex justify-between border-b border-gray-700
last:border-0 py-2">
            <span>{fan}</span>
            <span className="text-yellow-400"> 🏆 NFT</span>
          </li>
        )
      )
    </ul>
  </section>

  {/* Ostatnie wsparcia sekcja */}
  <section>
    <h2 className="font-semibold mb-2">Ostatnie wsparcia:</h2>
    <ul className="text-sm">
      <li>Janek wpłacił \ $5 - 5 min temu</li>
      <li>Anna wpłaciła \ $1 - 1 godz. temu</li>
      {/* ... kolejne wpisy */}
    </ul>
  </section>
</div>
);
}

```

- **Layout Komponentu:** Główny layout jest jednokolumnowy, co zapewnia doskonałą czytelność na urządzeniach mobilnych. W nagłówku zastosowano `flexbox` do horyzontalnego ułożenia awatara i danych tekstowych.
- **Stylizacja Kluczowych Elementów:** Awatar jest okrągły (`rounded-full`). Przycisk CTA zajmuje całą szerokość (`w-full`) i wykorzystuje akcentowy, złoty kolor (`bg-yellow-500`). Krytycznym wymogiem implementacyjnym jest zapewnienie, że **przycisk CTA jest zawsze dostępny**. Na urządzeniach mobilnych musi być on zrealizowany jako element "leпки" (`sticky`), stale widoczny na dole ekranu podczas przewijania. Na większych ekranach może powrócić do statycznej pozycji w layoutcie.

## Wytyczne Dostępności (WCAG)

- **Semantyczna Struktura HTML:** Nazwa twórcy musi być nagłówkiem najwyższego rzędu na tej stronie (<h1>), a tytuły sekcji (np. "Top Fani") nagłówkami niższego rzędu (<h2>). Zapewnia to logiczną strukturę dla technologii asystujących.
- **Alternatywy Tekstowe:** Awatar twórcy oraz wszelkie ikony (np. medale NFT) muszą posiadać odpowiednie teksty alternatywne (alt lub aria-label), które opisują ich zawartość lub funkcję.
- **Kontrast Kolorów:** Należy upewnić się, że wszystkie elementy tekstowe, w tym te o specjalnym znaczeniu (np. złoty tekst dla NFT), mają wystarczający kontrast w stosunku do tła.
- **Dostępność Modala:** Przycisk "Wesprzyj" aktywuje modal płatności. Modal ten musi być w pełni dostępny, co oznacza implementację "focus trap", obsługę zamykania klawiszem Esc oraz poprawne użycie atrybutów ARIA (role="dialog", aria-modal="true").

## 4.4 Panel Twórcy (Creator Dashboard)

Panel Twórcy to prywatne centrum zarządzania, zaprojektowane jako narzędzie pracy. Jego architektura musi być modułowa i skalowalna, aby w przyszłości można było łatwo dodawać nowe funkcje. Doświadczenie użytkownika jest zorientowane na szybki dostęp do kluczowych danych analitycznych, zarządzanie finansami i interakcję ze społecznością.

Struktura panelu opiera się na nawigacji bocznej i modułowej zawartości:

- **Menu Nawigacyjne:** Zawiera linki do kluczowych sekcji:
  - **Dashboard:** Główny widok z podsumowaniem najważniejszych informacji.
  - **Napiwki:** Szczegółowa historia wszystkich transakcji.
  - **Wypłaty:** Moduł do zarządzania wypłatami środków.
  - **Ustawienia:** Edycja profilu i ustawień konta.
- **Sekcja "Dashboard" (Podsumowanie):** Składa się z kluczowych widżetów:
  - **Saldo portfela:** Aktualny stan środków dostępnych do wypłaty.
  - **Szybkie statystyki:** Kafelki z kluczowymi metrykami (np. zarobki w miesiącu, liczba fanów).
  - **Wykres:** Wizualizacja przychodów w czasie.
  - **Ostatnie napiwki:** Skrócona lista najnowszych transakcji.

## Implementacja Techniczna (CreatorDashboard.tsx)

```
import React, { useState } from 'react';
import { motion } from 'framer-motion';

export function CreatorDashboard() {
  const [sidebarOpen, setSidebarOpen] = useState(false);

  return (
    <div className="flex min-h-screen bg-[#001F1F] text-white">
      {/* Sidebar (widoczny na md+, ukryty na mobile) */}
      <aside className={`bg-[#002B2B] w-64 p-4 space-y-4 \${sidebarOpen ?
"block" : "hidden"} md:block`}>
        <h2 className="text-lg font-bold mb-4">Menu</h2>
        <nav className="space-y-2">
          <a href="/dashboard" className="block px-2 py-1 rounded hover:bg-teal-700">Dashboard</a>
          <a href="/tips" className="block px-2 py-1 rounded hover:bg-teal-700">Napiwki</a>
          <a href="/withdraw" className="block px-2 py-1 rounded hover:bg-teal-700">Wypłaty</a>
          <a href="/settings" className="block px-2 py-1 rounded hover:bg-
```

```

teal-700">Ustawienia</a>
    {/* ...inne sekcje */}
</nav>
</aside>

{/* Przycisk hamburger (tylko mobile) */}
<button
  className="md:hidden p-2"
  onClick={() => setSidebarOpen(!sidebarOpen)}
  aria-label="Toggle menu">
  ≡
</button>

{/* Główna zawartość */}
<main className="flex-1 p-6">
  <h1 className="text-2xl font-bold mb-4">Cześć, [Imię]!</h1>

  {/* Szybkie statystyki - 3 kolumny na desktop, 1 kolumna na mobile
*/}

  <div className="grid grid-cols-1 sm:grid-cols-3 gap-4 mb-6">
    <div className="bg-gray-800 p-4 rounded">
      <p>Saldo portfela</p>
      <p className="text-2xl font-semibold">\$123.45</p>
    </div>
    <div className="bg-gray-800 p-4 rounded">
      <p>Napiwki (miesiąc)</p>
      <p className="text-2xl font-semibold">\$500</p>
    </div>
    <div className="bg-gray-800 p-4 rounded">
      <p>Unikalni fani</p>
      <p className="text-2xl font-semibold">42</p>
    </div>
  </div>

  {/* Wykres/diagram */}
  <div className="bg-gray-800 p-4 rounded mb-6">
    <h2 className="mb-2">Napiwki w tym tygodniu</h2>
    <div className="h-24 bg-gray-700"></div> {/* placeholder wykresu
*/}

  </div>

  {/* Ostatnie napiwki */}
  <div className="bg-gray-800 p-4 rounded mb-6">
    <h2 className="mb-2">Ostatnie napiwki</h2>
    <ul className="text-sm">
      <li className="border-b border-gray-700 py-1">Anna - \$5 - 2025-
08-01</li>
      <li className="border-b border-gray-700 py-1">Jan - \$10 - 2025-
08-01</li>

```

```

    { /* ... */ }
  </ul>
</div>

{ /* Akcje wymagające uwagi */ }
<div>
  <motion.div
    animate={{ opacity: [0.5, 1], scale: [0.98, 1] }}
    transition={{ repeat: Infinity, repeatType: "reverse",
duration: 2 }}
    className="bg-purple-700 p-4 rounded mb-2">
    Zweryfikuj swoje konto, aby otrzymywać wypłaty.
  </motion.div>
  { /* ...inne alerty */ }
</div>
</main>
</div>
);
}

```

- **Responsywny Layout:** Implementacja jest w pełni responsywna. Na urządzeniach mobilnych menu boczne (<aside>) jest domyślnie ukryte i kontrolowane przez przycisk "hamburger". Na ekranach o szerokości md (768px) i większych staje się ono stale widocznym elementem nawigacyjnym.
- **Siatka Statystyk (Grid):** Do stworzenia adaptacyjnej siatki dla kafelków ze statystykami użyto `display: grid`. Domyślnie jest ona jednokolumnowa, a na ekranach o szerokości sm i większych (`sm:grid-cols-3`) przełącza się na układ trójkolumnowy.
- **Dynamiczne Alerty:** Komponent `motion.div` z Framer Motion został użyty do stworzenia subtelnie pulsującego alertu. Animacja zmiany przezroczystości i skali w pętli skutecznie przyciąga uwagę twórcy do ważnych komunikatów.

## Wytyczne Dostępności (WCAG)

- **Nawigacja Klawiaturą:** Interfejs musi być w pełni obsługiwany za pomocą klawiatury, włączając w to otwieranie/zamykanie menu bocznego, nawigację po linkach i interakcję z elementami w głównej treści.
- **Role ARIA:** Boczne menu nawigacyjne musi być opakowane w semantyczny element `<nav>`, aby technologie asystujące mogły je poprawnie zinterpretować jako główny blok nawigacyjny strony.
- **Alternatywy dla Wykresów:** Dane prezentowane na wykresach muszą mieć swoją tekstową alternatywę. Może to być tabela z danymi dostępna pod wykresem lub ukryty opis tekstowy, który jest odczytywany przez czytniki ekranu.
- **Semantyczne Tabele:** Historia transakcji prezentowana w formie tabeli musi używać odpowiednich znaczników (`<table>`, `<thead>`, `<th>`, `<tbody>`), aby zapewnić poprawne odczytywanie relacji między komórkami a nagłówkami przez czytniki ekranu.

## 4.5 Panel Fana (Fan Dashboard)

Panel Fana to uproszczone centrum aktywności, zaprojektowane z myślą o osobach wspierających. Jego głównym celem jest budowanie zaangażowania i lojalności poprzez elementy grywalizacji, takie jak kolekcjonowanie odznak, oraz zapewnienie łatwego dostępu do historii wsparcia. Interfejs jest celowo prostszy niż panel twórcy, koncentrując się na doświadczeniach społecznościowych.

Główne sekcje Panelu Fana:

- **Lista wspieranych twórców:** Umożliwia fanowi szybki przegląd twórców, których wsparł, oraz łatwy powrót na ich profile.

- **Oznaki i osiągnięcia:** Galeria zdobytych odznak (NFT "Proof of Support"), która działa jako element grywalizacji i daje poczucie satysfakcji.
- **Opcja "Upgrade do Twórcy":** Wyraźnie widoczny przycisk lub baner zachęcający fana do stania się twórcą, co stanowi ważny element strategii wzrostu platformy.

## Implementacja Techniczna (FanPanel.tsx)

```
import React from 'react';

export function FanPanel() {
  return (
    <div className="max-w-2xl mx-auto p-4 text-white">
      <h1 className="text-2xl font-bold mb-4">Witaj, [Nick]!</h1>

      {/* Wspierani twórcy */}
      <section className="mb-6">
        <h2 className="text-xl font-semibold mb-2">Twoi wspierani
        Twórcy</h2>
        <ul className="space-y-2">
          <li className="bg-gray-800 p-3 rounded flex justify-between">
            <span>Twórca A</span>
            <button className="text-sm text-teal-400 underline">
              Przejdź do profilu
            </button>
          </li>
          {/* ... kolejne wspierane profile */}
        </ul>
      </section>

      {/* Odznaki */}
      <section className="mb-6">
        <h2 className="text-xl font-semibold mb-2">Odznaki wsparcia</h2>
        <div className="flex flex-wrap gap-2" role="list">
          <div className="bg-gray-800 p-2 rounded text-center"
          role="listitem">
            <span className="block text-2xl" aria-hidden="true">🏆</span>
            <span className="text-xs">#1 Fan Twórca A</span>
          </div>
          {/* ... kolejne odznaki */}
        </div>
      </section>

      {/* Historia wsparcia */}
      <section>
        <h2 className="text-xl font-semibold mb-2">Historia wsparcia</h2>
        <ul className="text-sm">
          <li>Twórca A - \ $5 - 2025-08-01</li>
          <li>Twórca B - \ $2 - 2025-07-20</li>
          {/* ... */}
        </ul>
      </section>
    </div>
  );
}
```

```

    </ul>
  </section>
</div>
);
}

```

- **Struktura Layoutu:** Komponent ma prostą, jednokolumnową strukturę, idealną dla urządzeń mobilnych. Poszczególne sekcje są jasno oddzielone za pomocą nagłówków `<h2>`, co zapewnia przejrzystość i łatwość nawigacji.
- **Prezentacja Odznak:** Do stworzenia responsywnej siatki na odznaki wykorzystano właściwości `flex` i `flex-wrap`. Dzięki temu odznaki automatycznie układają się w kolejnych wierszach, dopasowując się do dostępnej szerokości ekranu.

## Wytyczne Dostępności (WCAG)

- **Semantyczna Siatka Odznak:** Kontener z odznakami, mimo że jest zbudowany z `<div>`, musi mieć przypisaną rolę ARIA `role="list"`, a każdy kafelek z odznaką `role="listitem"`. Zapewnia to, że czytniki ekranu poprawnie zinterpretują tę sekcję jako kolekcję elementów.
- **Alternatywy Tekstowe dla Odznak:** Ponieważ odznaki NFT są elementami graficznymi, każda z nich musi posiadać opisowy tekst alternatywny (np. w atrybucie `alt` dla `<img>` lub `aria-label` dla `<div>`), który precyzyjnie opisuje, co dana odznaka reprezentuje (np. "Odznaka: #1 Fan dla Twórcy A").

## 4.6 Modal Napiwku (Tip Modal)

Modal Napiwku jest jednym z najbardziej krytycznych punktów całego procesu konwersji. Jego projekt musi być bezbłędny, szybki i budzący maksymalne zaufanie, aby zminimalizować liczbę porzuceń transakcji. Każdy element, od układu pól po mikrointerakcje, został zaprojektowany z myślą o płynnym i bezpiecznym doświadczeniu użytkownika.

Funkcjonalności modala obejmują:

- **Kwota wsparcia:** Umożliwia wybór predefiniowanej kwoty za pomocą przycisków szybkiego wyboru oraz wpisanie własnej, niestandardowej sumy.
- **Metoda płatności:** Oferuje wybór między tradycyjną kartą płatniczą a portfelem Web3, obniżając barierę wejścia dla różnych grup użytkowników.
- **Opcjonalna wiadomość:** Pole tekstowe, które pozwala fanowi na dodanie osobistej wiadomości do twórcy, co wzmacnia więź między nimi.
- **Przycisk finalizujący:** Wyraźny przycisk CTA, który inicjuje proces płatności. Modal musi także poprawnie obsługiwać zarówno użytkowników zalogowanych, jak i niezarejestrowanych gości.

## Implementacja Techniczna (TipModal.tsx)

```

import React, { useState } from 'react';
import { motion } from 'framer-motion';

export function TipModal({ onClose }) {
  const [amount, setAmount] = useState(5);
  const [method, setMethod] = useState(null); // "card" | "crypto" | etc.

  return (
    <div
      className="fixed inset-0 bg-black bg-opacity-50 flex items-center
        justify-center"
      role="dialog"
    >

```



```

aria-modal="true">
<motion.div
  initial={{ scale: 0.9, opacity: 0 }}
  animate={{ scale: 1, opacity: 1 }}
  className="bg-[#002B2B] p-6 rounded max-w-sm w-full">
<h3 className="text-xl font-semibold mb-4">Wesprzyj Twórcę</h3>

  {/* Kwota wsparcia */}
  <div className="mb-4">
    <div className="flex space-x-2 mb-2">
      {[1, 5, 10].map(val => (
        <button
          key={val}
          onClick={() => setAmount(val)}
          className={`px-3 py-1 rounded \${amount === val ? 'bg-teal-
600' : 'bg-gray-700'}}`}>
        \${val}
      </button>
      )]}
    </div>
    <label className="block">
      Inna kwota:
      <input
        type="number"
        min="1"
        className="w-full mt-1 p-2 rounded bg-gray-800 border border-
gray-600"
        value={amount}
        onChange={e => setAmount(Number(e.target.value))} />
    </label>
  </div>

  {/* Metoda płatności */}
  <div className="mb-4">
    <label className="block mb-1">Metoda płatności:</label>
    <div className="flex space-x-2">
      <button
        onClick={() => setMethod('card')}
        className={`flex-1 bg-gray-700 p-2 rounded \${method ===
'card' ? 'ring-2 ring-teal-500' : ''}}`>
        Karta
      </button>
      <button
        onClick={() => setMethod('crypto')}
        className={`flex-1 bg-gray-700 p-2 rounded \${method ===
'crypto' ? 'ring-2 ring-teal-500' : ''}}`>
        Crypto
      </button>
    </div>
  </div>

```



```

</div>

    {/* Wiadomość opcjonalna */}
    <div className="mb-4">
      <label className="block mb-1">Wiadomość (opcjonalnie):</label>
      <textarea
        className="w-full p-2 rounded bg-gray-800 border border-gray-
600"
        rows={3}
        placeholder="Napisz wiadomość dla twórcy...">
      </textarea>
    </div>

    <button className="w-full bg-yellow-500 text-gray-900 font-semibold
py-2 rounded hover:bg-yellow-600">
      Wyślij napiwek
    </button>
    <button onClick={onClose} className="mt-3 text-sm text-gray-400
underline">
      Anuluj
    </button>
  </motion.div>
</div>
);
}

```

- **Zarządzanie Stanem:** Hooki `useState` są używane do kontrolowania kluczowych danych wprowadzanych przez użytkownika, takich jak wybrana kwota (`amount`) i metoda płatności (`method`).
- **Interfejs Użytkownika:** Przyciskom szybkiego wyboru kwoty oraz metod płatności dynamicznie przypisywane są klasy CSS, które wizualnie wskazują aktywny wybór (zmiana koloru tła lub dodanie obramowania `ring`).
- **Dostępność Modala:** Komponent jest opakowany w `div` z atrybutami `role="dialog"` i `aria-modal="true"`. Informuje to technologie asystujące, że jest to okno dialogowe, które blokuje interakcję z resztą strony.

## Mikrointerakcje i Animacje

- **Animacja Wejścia:** Modal pojawia się z subtelną animacją skalowania i przezroczystości (`initial`, `animate`), co sprawia, że jego wejście jest płynne i mniej nachalne.
- **Walidacja z Efektem "Shake":** W przypadku błędu walidacji (np. wprowadzenie nieprawidłowej kwoty), pole formularza może wykonać krótką animację "wstrząśnięcia", aby wizualnie zasygnalizować problem.
- **Gratyfikacja po Transakcji:** Po pomyślnym zakończeniu płatności, kluczowe jest natychmiastowe, pozytywne wzmocnienie. Obligatoryjne jest wyświetlenie animacji konfetti, która celebrowie udane wsparcie i buduje pozytywne skojarzenia z procesem.

## Wytyczne Dostępności (WCAG)

- **Focus Trap:** Po otwarciu modala, fokus klawiatury musi być "uwięziony" w jego obrębie. Użytkownik nie może przypadkowo przenieść fokusu na elementy znajdujące się pod spodem.
- **Obsługa Klawisza Esc:** Użytkownik musi mieć możliwość zamknięcia modala za pomocą klawisza `Escape`. Po zamknięciu, fokus powinien powrócić do elementu, który pierwotnie otworzył modal.

- **Etykiety i Błędy:** Wszystkie pola muszą być powiązane z etykietami. Komunikaty o błędach walidacji muszą być jasne, tekstowe i programowo powiązane z odpowiednimi polami.
- **Kontrast CTA:** Przycisk finalizujący transakcję musi mieć wystarczający kontrast kolorystyczny. **Uwaga:** testy wykazały, że intensywnie żółty kolor tła (#FFD700) z czarnym tekstem ma niewystarczający kontrast (~1.13:1). Z tego powodu **obligatoryjne jest użycie ciemniejszego odcienia żółtego**, takiego jak tailwind-yellow-500 (#eab308), który zapewnia zgodność z WCAG.

## 4.7 Uwierzytelnianie (Logowanie i Rejestracja)

Moduł uwierzytelniania jest bramą do pełnego doświadczenia platformy. Nasza strategia polega na oferowaniu wielu metod logowania i rejestracji (e-mail, OAuth z Google/Twitch, portfel Web3), aby maksymalnie obniżyć barierę wejścia i dostosować się do preferencji różnych grup użytkowników.

### Implementacja Techniczna (AuthModal.tsx)

```
import React, { useState } from 'react';

export function AuthModal() {
  const [mode, setMode] = useState<'signup' | 'login'>('signup');

  return (
    <div
      className="fixed inset-0 bg-black bg-opacity-50 flex items-center justify-center"
      role="dialog"
      aria-modal="true">
      <div className="bg-[#002B2B] p-6 rounded max-w-sm w-full">
        {/* Przełącznik Rejestracja/Logowanie */}
        <div className="mb-4 flex justify-center space-x-4">
          <button
            onClick={() => setMode('signup')}
            className={`px-4 py-2 \${mode === 'signup' ? 'border-b-2 border-teal-400' : ''}`}>
            Rejestracja
          </button>
          <button
            onClick={() => setMode('login')}
            className={`px-4 py-2 \${mode === 'login' ? 'border-b-2 border-teal-400' : ''}`}>
            Logowanie
          </button>
        </div>

        {mode === 'signup' ? (
          <form>
            <label className="block mb-2">Email:
            <input
              type="email"
              className="w-full mt-1 p-2 rounded bg-gray-800 border border-gray-600"
              required />

```

```

        </label>
        <label className="block mb-4">Hasło:
          <input
            type="password"
            className="w-full mt-1 p-2 rounded bg-gray-800 border
border-gray-600"
            required />
        </label>
        <button type="submit" className="w-full bg-teal-600 hover:bg-
teal-700 py-2 rounded font-semibold mb-2">
          Zarejestruj się
        </button>
        <button
          type="button"
          className="w-full bg-gray-700 py-2 rounded mb-4"
          onClick={() => {/* OAuth Google */}}>
          Zarejestruj przez Google
        </button>
        <button
          type="button"
          className="w-full bg-gray-700 py-2 rounded mb-4"
          onClick={() => {/* OAuth Wallet */}}>
          Połącz portfel (Web3)
        </button>
        <p className="text-xs text-gray-400">
          Zakładając konto akceptujesz <a href="/terms"
className="underline">Regulamin</a>.
        </p>
      </form>
    ) : (
      <form>
        <label className="block mb-2">Email:
          <input
            type="email"
            className="w-full mt-1 p-2 rounded bg-gray-800 border
border-gray-600"
            required />
        </label>
        <label className="block mb-4">Hasło:
          <input
            type="password"
            className="w-full mt-1 p-2 rounded bg-gray-800 border
border-gray-600"
            required />
        </label>
        <button type="submit" className="w-full bg-teal-600 hover:bg-
teal-700 py-2 rounded font-semibold mb-4">
          Zaloguj się
        </button>
      </form>
    )
  }
}

```

```

    <button
      type="button"
      className="w-full bg-gray-700 py-2 rounded mb-4"
      onClick={() => {/* OAuth Google */}}>
      Zaloguj przez Google
    </button>
    <button
      type="button"
      className="w-full bg-gray-700 py-2 rounded mb-4"
      onClick={() => {/* OAuth Wallet */}}>
      Połącz portfel (Web3)
    </button>
    <p className="text-sm text-gray-300 text-center">
      Nie masz konta? <button
        type="button"
        onClick={() => setMode('signup')}
        className="underline">Zarejestruj się</button>
      </p>
    </form>
  )}
</div>
</div>
);
}

```

- **Przełączanie Trybów:** Stan mode (o wartościach 'signup' lub 'login') kontroluje, który formularz jest aktualnie renderowany. Zmiana stanu następuje po kliknięciu odpowiedniego przycisku-zakładki, co pozwala na płynne przełączanie się między rejestracją a logowaniem w ramach jednego komponentu.
- **Struktura Formularzy:** Oba formularze mają spójną strukturę, składającą się z pól input dla e-maila i hasła oraz zestawu przycisków dla różnych metod uwierzytelniania (własna, Google, Web3).

## Wytyczne Dostępności (WCAG)

- **Logiczna Kolejność Fokusu:** Sekwencja nawigacji klawiszem Tab musi przechodzić przez elementy formularza w logicznej kolejności: od przełącznika trybu, przez pola tekstowe, aż po przyciski akcji.
- **Powiązanie Etykiet z Polami:** Każde pole input musi być powiązane z widoczną etykietą <label>, co jest kluczowe dla użytkowników czytników ekranu.
- **Obsługa Błędów:** Komunikaty o błędach (np. "nieprawidłowe hasło") muszą być prezentowane w sposób dostępny. Oznacza to wyświetlenie tekstowego komunikatu i użycie atrybutów ARIA (np. aria-invalid) do programowego powiązania błędu z polem.
- **Dostępność Przełącznika:** Przełącznik trybu logowanie/rejestracja musi być w pełni obsługiwalny za pomocą klawiatury i jasno komunikować swój aktualny stan (np. poprzez wizualne podkreślenie i atrybuty ARIA).

## 5.0 Globalne Wytyczne i Dobre Praktyki

Oprócz szczegółowych specyfikacji dla poszczególnych komponentów, ostateczny sukces projektu zależy od konsekwentnego przestrzegania globalnych zasad spójności i unikania typowych pułapek projektowych. Poniższe wytyczne mają na celu zapewnienie, że cała aplikacja TipJar+ będzie miała profesjonalny, dopracowany i jednolity charakter.

## 5.1 Checklista Spójności UI/UX

Poniższa checklista musi być stosowana przez deweloperów podczas implementacji każdego nowego elementu interfejsu.

- ☐ [ ] **Stosuj wyłącznie fonty, kolory i ikony zdefiniowane w systemie designu.**
- ☐ [ ] **Zapewnij, że przyciski, pola formularzy i inne elementy interaktywne mają ten sam styl i zachowanie (hover, focus, active) co w innych częściach aplikacji.**
- ☐ [ ] **Implementuj layouty zgodnie z ustalonymi wzorcami (np. rozmieszczenie nawigacji, nagłówków).**
- ☐ [ ] **Używaj spójnej terminologii (np. "Napiwek") w całej aplikacji i utrzymuj jednolity, przyjazny ton komunikacji.**
- ☐ [ ] **Zawsze sprawdzaj, czy nowy element nie duplikuje funkcjonalności istniejącego, reużywalnego komponentu.**
- ☐ [ ] **Gwarantuj, że doświadczenie użytkownika jest logicznie spójne na urządzeniach mobilnych i desktopowych.**
- ☐ [ ] **Implementuj informację zwrotną dla użytkownika (np. animacja błędu, komunikat o sukcesie) w ten sam sposób, co w innych miejscach.**

## 5.2 Najczęstsze Błędy do Uniknięcia

- **Przeładowanie interfejsu:** Unikaj umieszczania zbyt wielu informacji na jednym ekranie. Zbyt duża gęstość elementów przytłacza użytkownika. Stosuj zasadę progresywnego ujawniania, np. na profilu twórcy wyświetlaj Top 5 fanów z linkiem do pełnej listy, zamiast całej listy od razu.
- **Priorytet stylu nad użytecznością:** Estetyka nigdy nie może stać ponad funkcjonalnością. Efektowne animacje czy tła nie mogą pogarszać czytelności tekstu (kontrastu) ani utrudniać interakcji.
- **Brak informacji zwrotnej:** Użytkownik musi zawsze otrzymywać natychmiastową informację o wyniku swoich działań. Brak reakcji systemu po kliknięciu przycisku (np. przez brak stanu loading) prowadzi do frustracji i niepewności.
- **Ignorowanie standardów i oczekiwań:** Nie należy na siłę wymyślać koła na nowo. Stosowanie powszechnie znanych wzorców interakcji (np. ikona kosza do usuwania) skraca czas potrzebny na naukę obsługi aplikacji.
- **Niejasne komunikaty:** Komunikaty o błędach i instrukcje muszą być napisane prostym i zrozumiałym dla każdego językiem. **Szczególnie w kontekście Web3, należy bezwzględnie unikać technicznego żargonu bez wyjaśnienia.**
- **Zbyt nachalne animacje:** Mikrointerakcje powinny być subtelne i wspierać użyteczność, a nie rozpraszać. Długie, blokujące interfejs animacje są szkodliwe dla doświadczenia użytkownika.

Rygorystyczne przestrzeganie zasad zawartych w tej dokumentacji nie jest sugestią, lecz warunkiem koniecznym do stworzenia produktu światowej klasy. Naszym celem jest zbudowanie platformy, która wyznaczy nowe standardy w dziedzinie wspierania twórców, a spójność i dbałość o szczegóły są fundamentem tego sukcesu.